

Ambiente virtualizado — passo a passo

Pré-requisitos:

- Ambiente *Linux* com *QEMU* / *KVM*
- *ISO* do *Ubuntu Server 24.04*

Instalar o QEMU:

```
sudo apt install qemu-system-x86-64 qemu-utils ovmf
```

Criar e ativar a bridge br0:

```
sudo ip link add name br0 type bridge
sudo ip link set br0 up
```

Clonar o maas-dev-setup:

```
git clone https://github.com/canonical/maas-dev-setup
cd maas-dev-setup
```

Commit testado: dfcda058.

Configurar o repositório do MAAS:

O `setup-dev-env.sh` clona o repositório oficial por padrão. Para usar o *fork* com as modificações da extensão, alterar a *URL* no *script*:

```
# Em setup-dev-env.sh, substituir:
# git clone --origin upstream https://github.com/canonical/maas.git .
# por:
# git clone --origin upstream -b hackingmaas https://github.com/EGmux/maas.git .
```

Criar a VM controladora:

```
qemu-system-x86_64 \
  -accel kvm \
  -cpu host,migratable=off,+kvm-pv-unhalt,+kvm-pv-eoi \
  -object memory-backend-ram,id=mem0,size=2048M,prealloc=yes \
  -boot order=c,menu=off,strict=off \
  -m size=4096M \
  -smp 8,sockets=1,dies=1,cores=8,threads=1 \
  -device virtio-blk-pci,drive=disk_data \
  -device virtio-net-pci,netdev=controllernet \
  -device virtio-net-pci,netdev=clientnet \
  -name maas-controller \
  -blockdev driver=file,filename=maas-controller.qcow2,node-
name=maasimg,discard=unmap,detect-zeroes=unmap \
  -blockdev driver=qcow2,file=maasimg,node-name=disk_data \
  -blockdev driver=file,node-name=installer_iso,filename=ubuntu-26.04-live-server-
amd64.iso,read-only=on \
  -blockdev driver=raw,node-name=installer,file=installer_iso \
  -device ide-cd,drive=installer \
  -display spice-app,gl=on \
  -spice port=5900,disable-ticketing=off \
  -chardev spicevmc,id=vdagent,name=vdagent \
  -device virtio-serial-pci \
  -device virtserialport,chardev=vdagent,name=com.redhat.spice.0 \
  -netdev passt,id=controllernet,ipv6=off,tcp-ports=8022:22,tcp-ports=5240:5240 \
  -netdev bridge,br=br0,id=clientnet \
  -vga qxl
```

Após a instalação, a controladora estará acessível via *SSH* na porta 8022 do *host* (`ssh -p 8022 user@localhost`) e a *API* do *MAAS* na porta 5240.

Criar chave SSH:

```
ssh-keygen -t rsa -f ~/.ssh/id_rsa
ssh-copy-id -i ~/.ssh/id_rsa.pub -p 8022 user@localhost
```

Encaminhar porta para o contêiner LXD:

O encaminhamento de porta foi feito via *LXD*, não via *iptables*. O comando utilizado foi equivalente a adicionar um *device proxy* ao contêiner para expor a porta 5240 da *API* do *MAAS*:

```
# Adicionar proxy LXD para a API do MAAS
lxc config device add maas-dev maas-api proxy \
    listen=tcp:0.0.0.0:5240 \
    connect=tcp:127.0.0.1:5240
```

Acoplar a interface da VM à bridge maas-kvm:

```
sudo ip link set dev ens5 master maas-kvm
```

Criar a VM cliente:

Antes de iniciar a *VM*, copiar o arquivo de variáveis *UEFI* do *OVMF*:

```
mkdir -p testvm1
cp /usr/share/OVMF/OVMF_VARS_4M.fd testvm1/OVMF_VARS_4M.fd
qemu-system-x86_64 \
    -machine type=q35,accel=kvm \
    -boot order=n \
    -drive if=pflash,format=raw,readonly=on,file=/usr/share/OVMF/OVMF_CODE_4M.fd \
    -drive if=pflash,format=raw,file=testvm1/OVMF_VARS_4M.fd \
    -m size=2048M \
    -smp 4 \
    -object rng-random,id=objrng0,filename=/dev/urandom \
    -device virtio-rng-pci,rng=objrng0 \
    -device virtio-blk-pci,drive=disk_data \
    -device virtio-net-pci,netdev=clientnet,bootindex=0,romfile=efi-virtio.rom \
    -name maas-testvm1 \
    -blockdev driver=file,filename=maas-testvm1.qcow2,node-
name=vmimg,discard=unmap,detect-zeroes=unmap \
    -blockdev driver=qcow2,file=vmimg,node-name=disk_data \
    -display spice-app,gl=on \
    -spice port=5900,disable-ticketing=off \
    -chardev spicevmc,id=vdagent,name=vdagent \
    -device virtio-serial-pci \
    -device virtserialport,chardev=vdagent,name=com.redhat.spice.0 \
    -netdev bridge,br=br0,id=clientnet \
    -vga qxl
```

A *VM* cliente faz *PXE boot* na rede *br0*, recebe *IP* via *DHCP* do *MAAS* e inicia o processo de *commissioning*.

Verificar PXE boot:

Se configurado corretamente, ao ligar a *VM* cliente, ela faz *PXE boot*, recebe *IP* via *DHCP* do *rackd* e inicia o comissionamento no *MAAS*.

```
maas admin machines read | jq '.[].status_name'
```